
Clasificación de movimientos de rehabilitación

Reto REHAB

Solución basada en señales cinemáticas y aprendizaje automático

Curso: TC3009C – Concentración de Inteligencia Artificial Avanzada para la Ciencia de Datos

Integrantes del equipo

Getzemani Kinari Ríos Tavares – A00573866

Diego Michell Villa Durán – A00836723

Emiliano Guzmán Ibarrola – A01639450

Resumen ejecutivo

Este proyecto desarrolla una solución de clasificación multiclase para reconocer 16 movimientos de entrenamiento de rehabilitación a partir de señales cinemáticas del conjunto REHAB. Se trabajó con los archivos procesados de dos grupos de sensores, se reconstruyó un archivo corrupto a partir de su versión cruda, se realizó un análisis exploratorio y se diseñó un proceso reproducible de limpieza, partición, creación de ventanas y extracción de características.

Cada repetición combina dos registros sincronizados de dimensiones 880×6 para obtener una señal de 880×12 . Las señales completas se dividieron de forma estratificada en 60% entrenamiento, 20% validación y 20% prueba antes de formar cuatro ventanas de 220 puntos, evitando que fragmentos de la misma repetición aparecieran en conjuntos distintos. Para cada ventana se calcularon diez estadísticas por canal, generando 120 características.

Se compararon regresión logística, SVM y Random Forest. Random Forest obtuvo el mejor F1 macro en validación (0.9443). Se detectó un ligero sobreajuste y se evaluaron 16 combinaciones mediante una búsqueda manual en cuadrícula, variando la profundidad máxima y los mínimos de muestras por división y hoja. Las configuraciones más restrictivas no mejoraron la validación, por lo que se conservó la configuración inicial. El modelo definitivo, reajustado con entrenamiento y validación, alcanzó en prueba 95.63% de exactitud, 95.49% de precisión macro y 95.31% de F1 macro.

Palabras clave: accidente cerebrovascular, rehabilitación, sensores portátiles, clasificación multiclase, Random Forest, señales cinemáticas.

Índice

1	Introducción	1
1.1	Planteamiento del problema	1
1.2	Objetivo general	1
1.3	Objetivos específicos	1
2	Datos y estructura de las señales	1
3	Reconstrucción del archivo corrupto	2
4	Análisis exploratorio de datos	2
4.1	Cantidad de repeticiones	2
4.2	Comportamiento temporal y distribuciones	3
4.3	Valores faltantes, ceros y duplicados	4
4.4	Valores extremos y relaciones entre canales	4
5	ETL y preparación para el modelo	6
5.1	Limpieza y unión de sensores	6
5.2	Partición estratificada y agrupada	6
5.3	Ventanas y extracción de características	7
5.4	Tratamiento del desbalance	7
6	Modelación	8
6.1	Modelos evaluados	8
6.2	Métricas	8
6.3	Comparación en validación	8
7	Refinamiento y regularización	8
7.1	Grid Search manual	8
7.2	Curva de validación	9
8	Evaluación final	10
9	Interfaz web	11
10	Limitaciones	12
11	Conclusiones	12
12	Contribuciones del equipo	13
A	Reproducibilidad	13
B	Referencia de formas	13

1 Introducción

El accidente cerebrovascular puede producir parálisis, debilidad muscular y problemas de coordinación que requieren rehabilitación prolongada. La evaluación objetiva de los ejercicios es importante para dar seguimiento a la recuperación y, en escenarios domiciliarios, puede beneficiarse de sensores portátiles que registren el movimiento sin utilizar cámaras.

REHAB es un conjunto de datos cinemáticos recolectado con sensores portátiles en 120 pacientes después de un accidente cerebrovascular. Incluye 27 movimientos de evaluación y 16 movimientos de entrenamiento estandarizados [1]. Por el alcance académico y el tiempo disponible, este proyecto se concentra exclusivamente en los 16 movimientos de entrenamiento y en los archivos de `d02_processed_data`.

1.1 Planteamiento del problema

La pregunta central es: *¿es posible identificar automáticamente cuál de las 16 actividades de entrenamiento se está realizando a partir de las señales de los sensores?* Esto corresponde a un problema de aprendizaje supervisado y clasificación multiclase. La entrada son mediciones cinemáticas multicanal y la salida es una etiqueta entre 000 y 015.

1.2 Objetivo general

Construir y evaluar una solución reproducible que transforme señales cinemáticas en características estadísticas y clasifique las 16 actividades de entrenamiento de REHAB, manteniendo una separación rigurosa entre entrenamiento, validación y prueba.

1.3 Objetivos específicos

- Comprender la estructura, distribución y calidad de los datos procesados.
- Reconstruir y validar el archivo corrupto `014_1.npy`.
- Limpiar señales vacías y duplicadas sin introducir fuga de información.
- Comparar modelos compatibles con datos tabulares numéricos.
- Analizar sobreajuste y refinar Random Forest mediante regularización estructural.
- Implementar una interfaz web para utilizar y explicar el modelo definitivo.

2 Datos y estructura de las señales

Se utilizaron 32 archivos NumPy: dos archivos por cada una de las 16 actividades. La nomenclatura tiene la forma `actividad_grupo.npy`; por ejemplo, `003_1.npy` y `003_2.npy` contienen los dos grupos de sensores para la actividad 003.

Cada archivo tiene forma $(n, 880, 6)$, donde n es el número de repeticiones, 880 es la cantidad de puntos temporales y 6 es el número de canales del grupo. La repetición i del archivo `003_1.npy` corresponde a la misma ejecución que la repetición i de `003_2.npy`. Ambas se concatenan por columnas:

$$(880 \times 6)_{\text{grupo 1}} + (880 \times 6)_{\text{grupo 2}} \longrightarrow (880 \times 12)_{\text{repetición completa}}. \quad (1)$$

Todas las repeticiones de ese par conservan la etiqueta 003. Los canales se identificaron como `pitch1`, `yaw1`, `roll1`, `pitch2`, `yaw2`, `roll2`, `f1-f5` y `pitch3`.

Cuadro 1: Estructura principal del conjunto utilizado.

Elemento	Descripción
Actividades	16 etiquetas, de 000 a 015
Archivos procesados	32; dos por actividad
Estructura por archivo	$(n, 880, 6)$
Repetición combinada	$(880, 12)$
Unidad de modelado	Ventana de $(220, 12)$
Variable objetivo	Etiqueta de actividad

3 Reconstrucción del archivo corrupto

El archivo procesado `014_1.npy` no podía cargarse porque contenía información binaria inválida. Dado que su versión cruda estaba disponible, se reconstruyó aplicando el procedimiento de preprocesamiento documentado por los autores: filtro de media móvil de 10 puntos, conservando la longitud, y normalización a media cero por repetición y canal.

Antes de utilizar el resultado, el procedimiento se validó con seis pares raw-processed oficiales: `000_1`, `000_2`, `001_1`, `001_2`, `013_1` y `013_2`. Todas las reconstrucciones conservaron su forma y cumplieron `np.allclose(rtol=1e-10, atol=1e-10)`. Las diferencias máximas fueron inferiores a 9×10^{-13} , atribuibles al redondeo de punto flotante.

Después se aplicó la transformación a `014_1.npy`. El archivo reconstruido tiene forma $(359, 880, 6)$, no contiene valores faltantes ni infinitos y fue incorporado a la carpeta procesada para que el ETL cargara los 32 archivos de manera uniforme.

Resultado de la reconstrucción. El archivo no se “rellenó” ni se estimó a partir de otras actividades. Se volvió a generar desde su señal raw utilizando y validando el procesamiento de los autores.

4 Análisis exploratorio de datos

El EDA se realizó para comprender la estructura y detectar problemas que pudieran afectar el clasificador. Se revisaron formas, cantidad de repeticiones, distribuciones, valores faltantes, infinitos, ceros, duplicados, valores extremos y relaciones entre canales.

4.1 Cantidad de repeticiones

La cantidad de repeticiones varía entre actividades, por lo que existe un desbalance moderado. La actividad 007 presenta la mayor cantidad original, mientras que otras, como 001, tienen menos observaciones. Esta diferencia justificó una partición estratificada, métricas macro y pesos balanceados durante el entrenamiento de Random Forest.

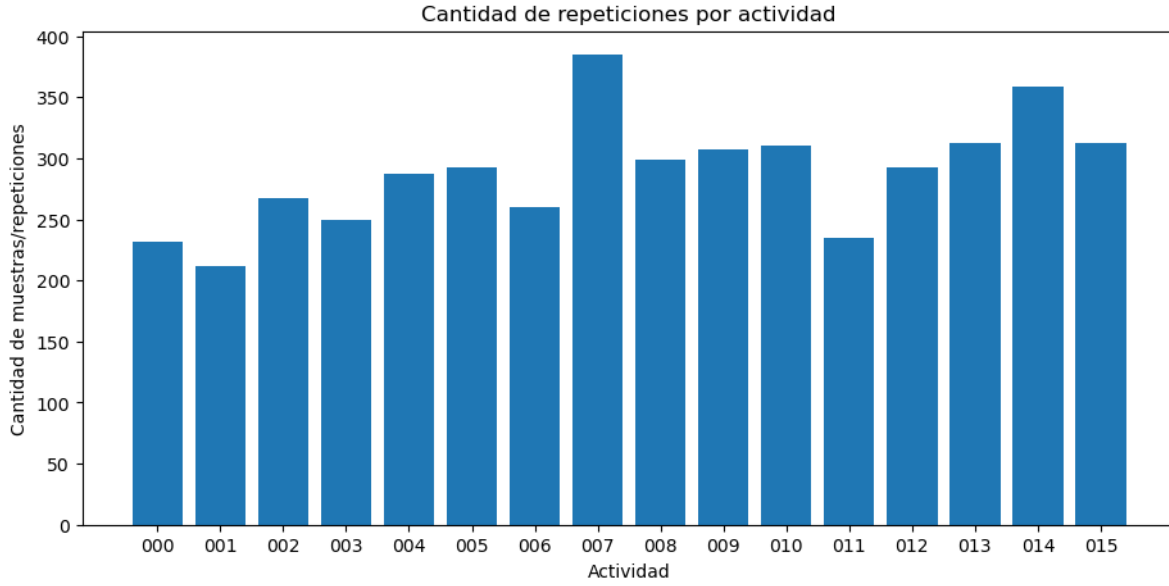


Figura 1: Cantidad original de repeticiones disponibles por actividad.

4.2 Comportamiento temporal y distribuciones

Una repetición no es un único valor: contiene 880 observaciones sucesivas para cada canal. La Figura 2 muestra cómo cambia un canal durante el movimiento. Estas variaciones justifican conservar información estadística sobre nivel, dispersión, amplitud y energía.

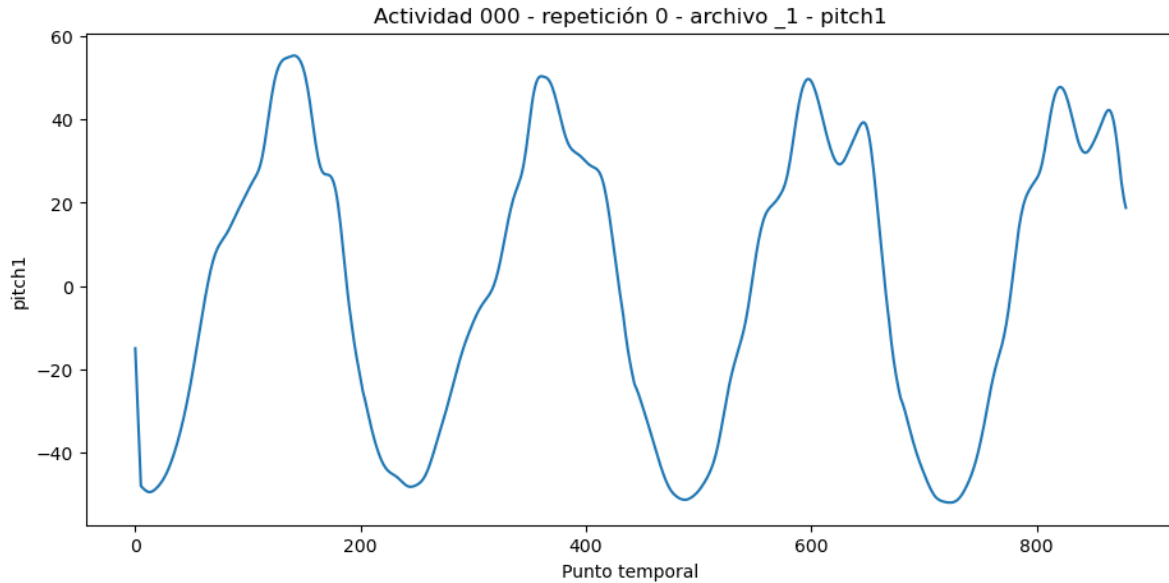


Figura 2: Ejemplo del comportamiento temporal de un canal en una repetición.

Los histogramas permitieron comparar actividades y observar diferencias en localización, dispersión y forma. No se conservaron todos los histogramas en este reporte para evitar redundancia; se presenta un ejemplo representativo en la Figura 3.

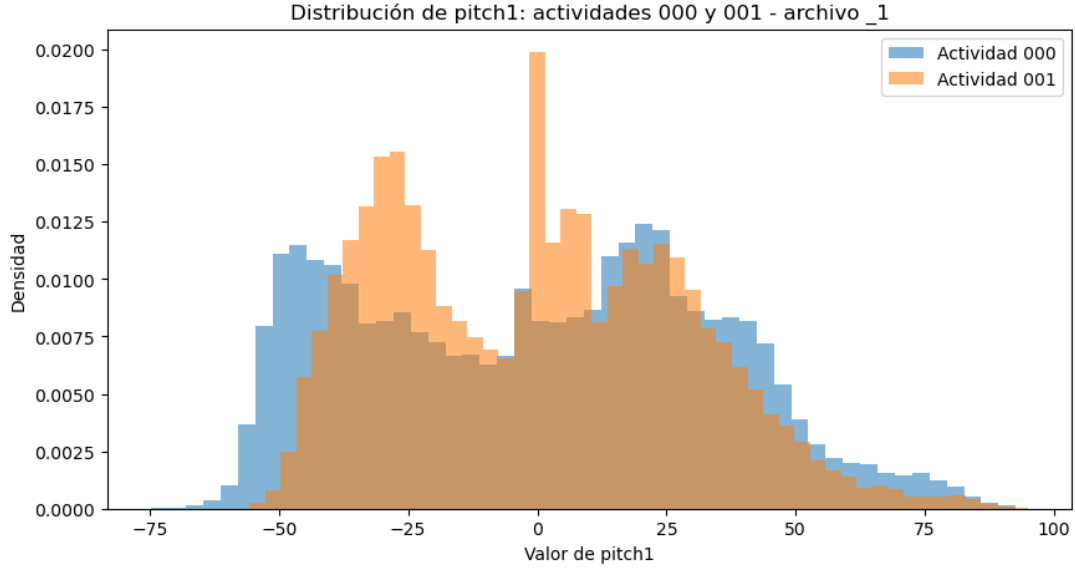


Figura 3: Comparación de la distribución de un canal entre dos actividades.

4.3 Valores faltantes, ceros y duplicados

No se encontraron valores NaN ni infinitos. El problema de calidad fueron las repeticiones completamente en cero. Después de unir los grupos, se eliminaron únicamente las señales cuyos 12 canales estaban en cero. Se conservaron seis repeticiones donde un grupo estaba en cero pero el otro todavía contenía información.

Se detectaron 1,235 muestras pertenecientes a algún grupo duplicado, organizadas en 577 grupos. Al conservar una observación por grupo se eliminaron 658 copias adicionales. Ningún grupo duplicado contenía etiquetas diferentes, por lo que la limpieza no eliminó contradicciones entre actividades.

Cuadro 2: Resumen de calidad y limpieza.

Indicador	Resultado
Señales combinadas iniciales	4,616
Señales completamente en cero	68
Copias duplicadas adicionales	658
Grupos distintos de duplicados	577
Duplicados con etiquetas diferentes	0
Señales limpias finales	3,890

4.4 Valores extremos y relaciones entre canales

El boxplot de $f1$ permitió observar dispersión y valores extremos entre actividades. Los valores extremos no se eliminaron automáticamente, porque pueden representar variabilidad real de los movimientos o de la condición motora de los participantes.

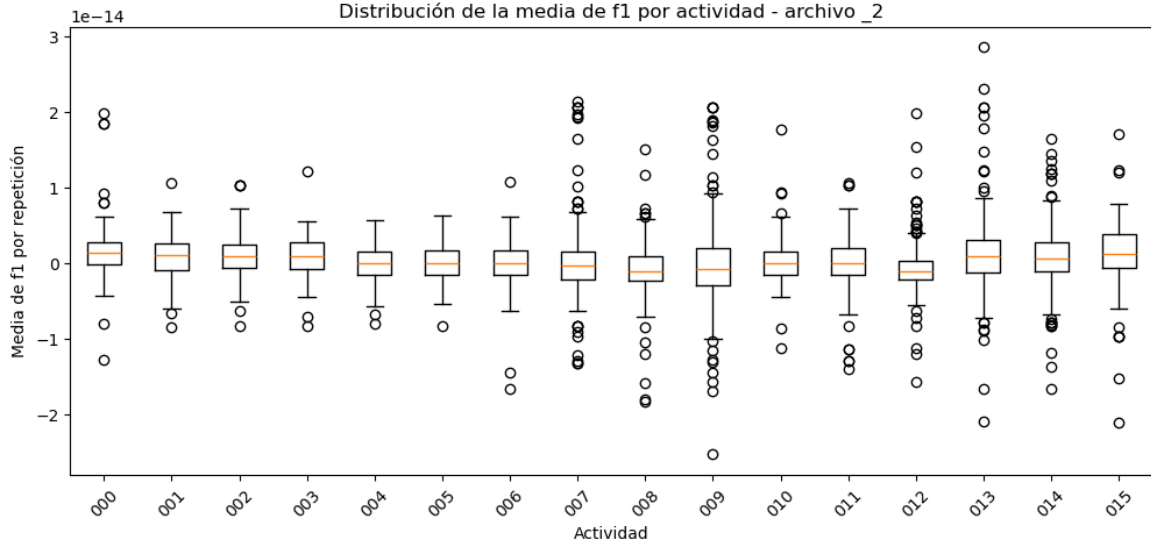


Figura 4: Distribución de la característica $f1$ por actividad.

El análisis bivariado mostró que la relación entre canales puede variar por actividad. Las matrices de correlación se calcularon agregando las 16 actividades por grupo de sensores; se usaron como descripción, no como criterio automático para descartar canales.

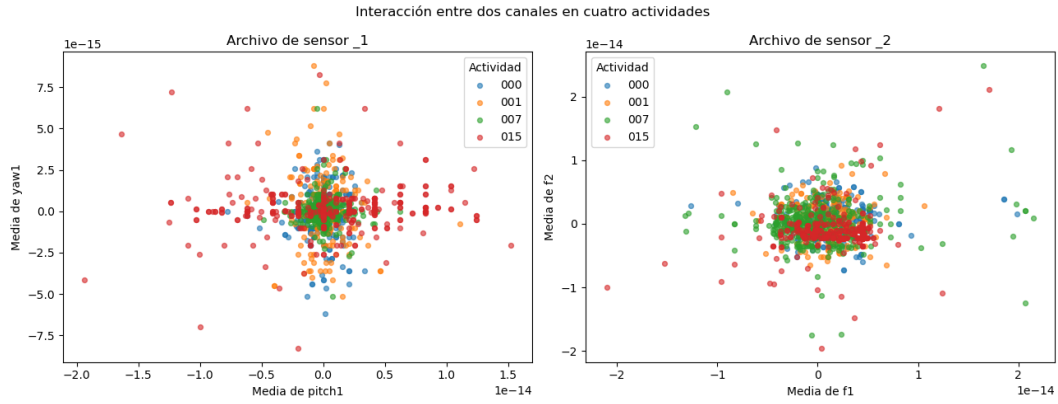
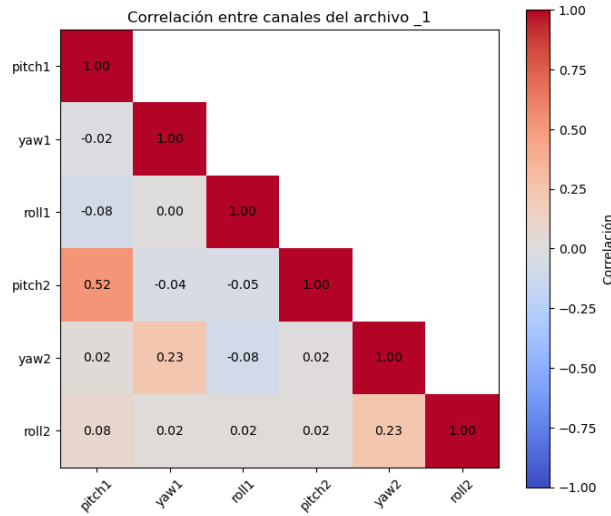
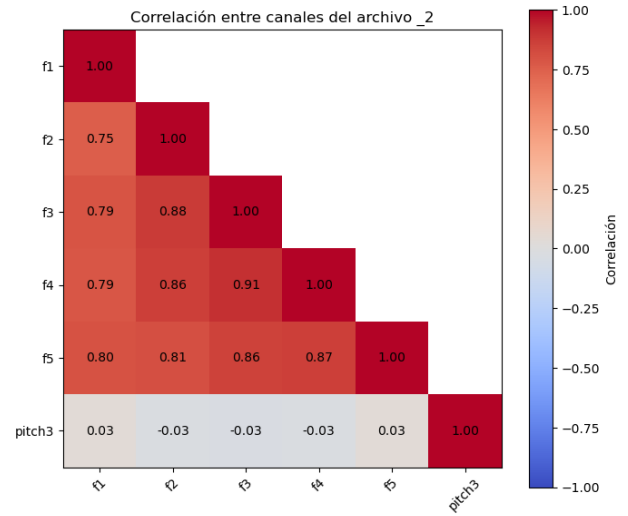


Figura 5: Ejemplo de interacción entre dos canales y actividades.



(a) Grupo de sensores 1.



(b) Grupo de sensores 2.

Figura 6: Correlaciones lineales entre canales para los dos grupos de sensores.

5 ETL y preparación para el modelo

5.1 Limpieza y unión de sensores

El ETL carga los 32 archivos, verifica su estructura y empareja las repeticiones de los grupos _1 y _2. Después elimina señales totalmente vacías y duplicados exactos considerando simultáneamente los 880 puntos y los 12 canales.

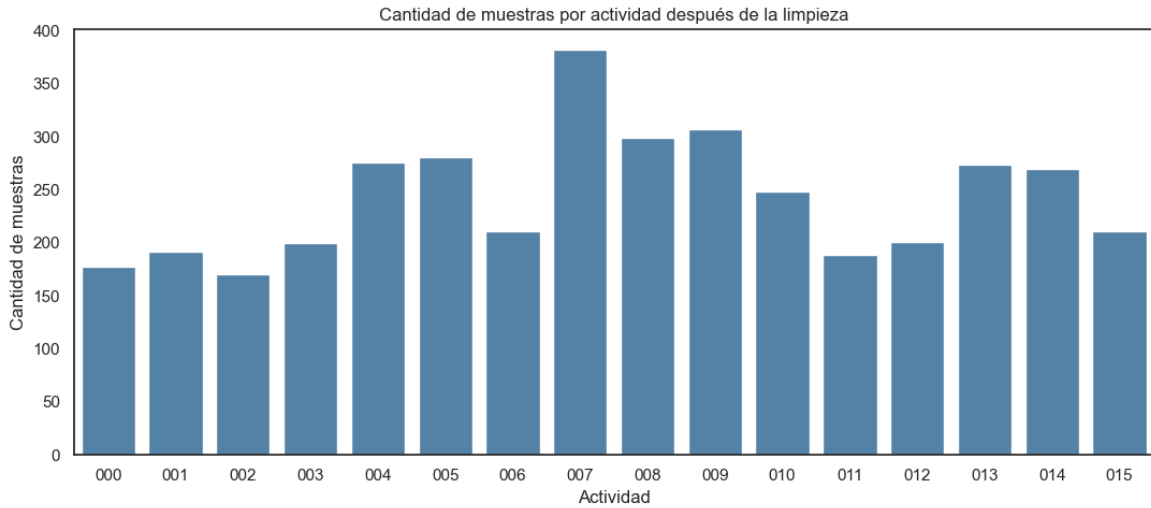


Figura 7: Cantidad de señales por actividad después de eliminar ceros completos y duplicados.

5.2 Partición estratificada y agrupada

La partición se hizo a nivel de señal completa, antes de generar ventanas. De este modo, todas las ventanas de una misma repetición permanecen en un único conjunto. Esta decisión evita fuga de información entre entrenamiento, validación y prueba.

3,890 señales completas $\rightarrow$ partición estratificada 60/20/20 $\rightarrow$ ventanas dentro de cada conjunto $\rightarrow$ características estadísticas

Cuadro 3: Tamaños antes y después de crear ventanas.

Conjunto	Señales completas	Ventanas	Porcentaje
Entrenamiento	2,334	9,336	60%
Validación	778	3,112	20%
Prueba	778	3,112	20%

La estratificación mantiene proporciones semejantes de las 16 actividades. El conjunto de validación se reservó para comparar modelos y seleccionar hiperparámetros; prueba no participó hasta la evaluación final.

5.3 Ventanas y extracción de características

Cada señal de 880 puntos se dividió en cuatro ventanas consecutivas de 220 puntos, sin traslape. Cada ventana conserva la etiqueta de la repetición original. Para cada uno de los 12 canales se calcularon:

1. media,
2. mediana,
3. desviación estándar,
4. mínimo,
5. máximo,
6. rango,
7. percentil 25,
8. percentil 75,
9. rango intercuartílico y
10. RMS.

Por tanto, cada ventana se transforma de $(220, 12)$ en una fila de $10 \times 12 = 120$ características. La transformación se aplicó de forma idéntica a entrenamiento, validación, prueba y entradas nuevas de la interfaz.

5.4 Tratamiento del desbalance

No se aplicaron SMOTE, sobremuestreo ni submuestreo porque el desbalance era moderado y generar ventanas sintéticas podía introducir patrones poco representativos. Se utilizaron tres medidas complementarias:

- partición estratificada para conservar proporciones;
- `class_weight=balanced` durante el entrenamiento de Random Forest;
- precisión macro y F1 macro para dar el mismo peso a cada actividad al evaluar.

6 Modelación

6.1 Modelos evaluados

Las características finales son tabulares, numéricas y no requieren conservar la estructura temporal original. Se compararon tres enfoques:

- **Regresión logística:** referencia lineal e interpretable.
- **SVM con kernel RBF:** fronteras no lineales en el espacio de características.
- **Random Forest:** relaciones no lineales e interacciones sin requerir escalamiento.

Regresión logística y SVM utilizaron estandarización dentro de sus respectivos pipelines. Random Forest no la necesitó por estar basado en divisiones de árboles.

6.2 Métricas

Se reportaron exactitud, precisión macro y F1 macro. La exactitud resume la proporción total de aciertos; la precisión macro evalúa la confiabilidad promedio de las predicciones dando el mismo peso a cada clase; el F1 macro equilibra precisión y recall para las 16 actividades. La matriz de confusión permite identificar pares específicos de actividades confundidas.

6.3 Comparación en validación

Todos los modelos se ajustaron únicamente con entrenamiento y se compararon en el conjunto fijo de validación.

Cuadro 4: Comparación de modelos en validación.

Modelo	Exactitud	Precisión macro	F1 macro
Regresión logística	0.7751	0.7640	0.7636
SVM	0.8602	0.8539	0.8524
Random Forest	0.9460	0.9453	0.9443

Random Forest fue seleccionado porque obtuvo el mayor resultado en las tres métricas y mantuvo un desempeño equilibrado entre actividades.

7 Refinamiento y regularización

El Random Forest inicial alcanzó F1 macro de 1.0000 en entrenamiento y 0.9443 en validación. La brecha de 0.0557 se interpretó como un ligero sobreajuste. Para estudiar si una versión menos compleja generalizaba mejor, se realizó una búsqueda manual en cuadrícula.

7.1 Grid Search manual

Se evaluaron todas las combinaciones de tres hiperparámetros de regularización estructural:

Cuadro 5: Cuadrícula de hiperparámetros evaluada.

Hiperparámetro	Valores	Efecto
max_depth	None, 15, 25, 30	Profundidad máxima
min_samples_split	2, 5	Mínimo para dividir
min_samples_leaf	1, 2	Mínimo por hoja

La cuadrícula contiene $4 \times 2 \times 2 = 16$ configuraciones. Cada una se entrenó con train y se evaluó en validation mediante F1 macro. No se utilizó `GridSearchCV`, porque existía un conjunto independiente de validación y no se realizó validación cruzada.

7.2 Curva de validación

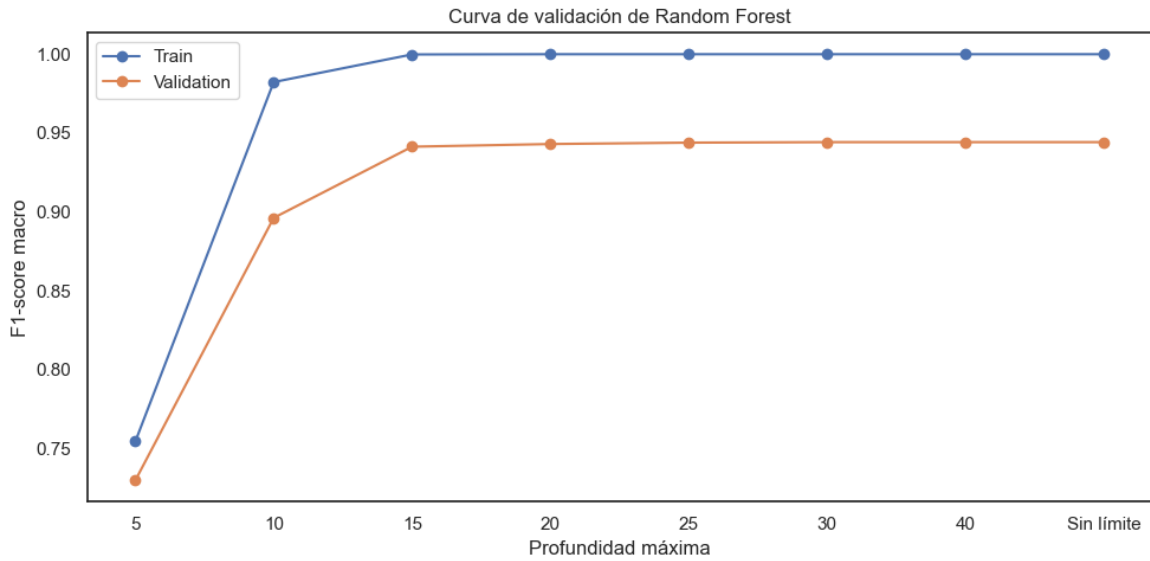


Figura 8: F1 macro de entrenamiento y validación según la profundidad máxima.

Con profundidad 5, ambos resultados fueron bajos, lo que indica subajuste. Al aumentar la profundidad, validación mejoró y se estabilizó alrededor de 0.9443; no disminuyó con árboles más profundos. Las profundidades 30, 40 y sin límite produjeron el mismo resultado mostrado. Limitar la profundidad o aumentar los mínimos de división y hoja no mejoró la generalización.

La mejor configuración entre las 16 evaluadas coincidió con el modelo inicial:

Cuadro 6: Configuración definitiva de Random Forest.

Hiperparámetro	Valor final
n_estimators	200
max_depth	None (sin límite)
min_samples_split	2
min_samples_leaf	1
max_features	sqrt
class_weight	balanced
random_state	42

La búsqueda fue útil aunque no cambiara el modelo: confirmó de forma sistemática que ninguna alternativa evaluada superaba la configuración inicial. No se afirma que sea la mejor configuración posible, sino la mejor entre las 16 combinaciones consideradas.

8 Evaluación final

Después de seleccionar los hiperparámetros, entrenamiento y validación se unieron. Se creó un nuevo Random Forest desde cero, se ajustó con las 12,448 ventanas de desarrollo y se evaluó una sola vez sobre las 3,112 ventanas de prueba.

Cuadro 7: Desempeño final sobre el conjunto de prueba reservado.

Métrica	Resultado
Exactitud	0.9563 (95.63%)
Precisión macro	0.9549 (95.49%)
F1 macro	0.9531 (95.31%)

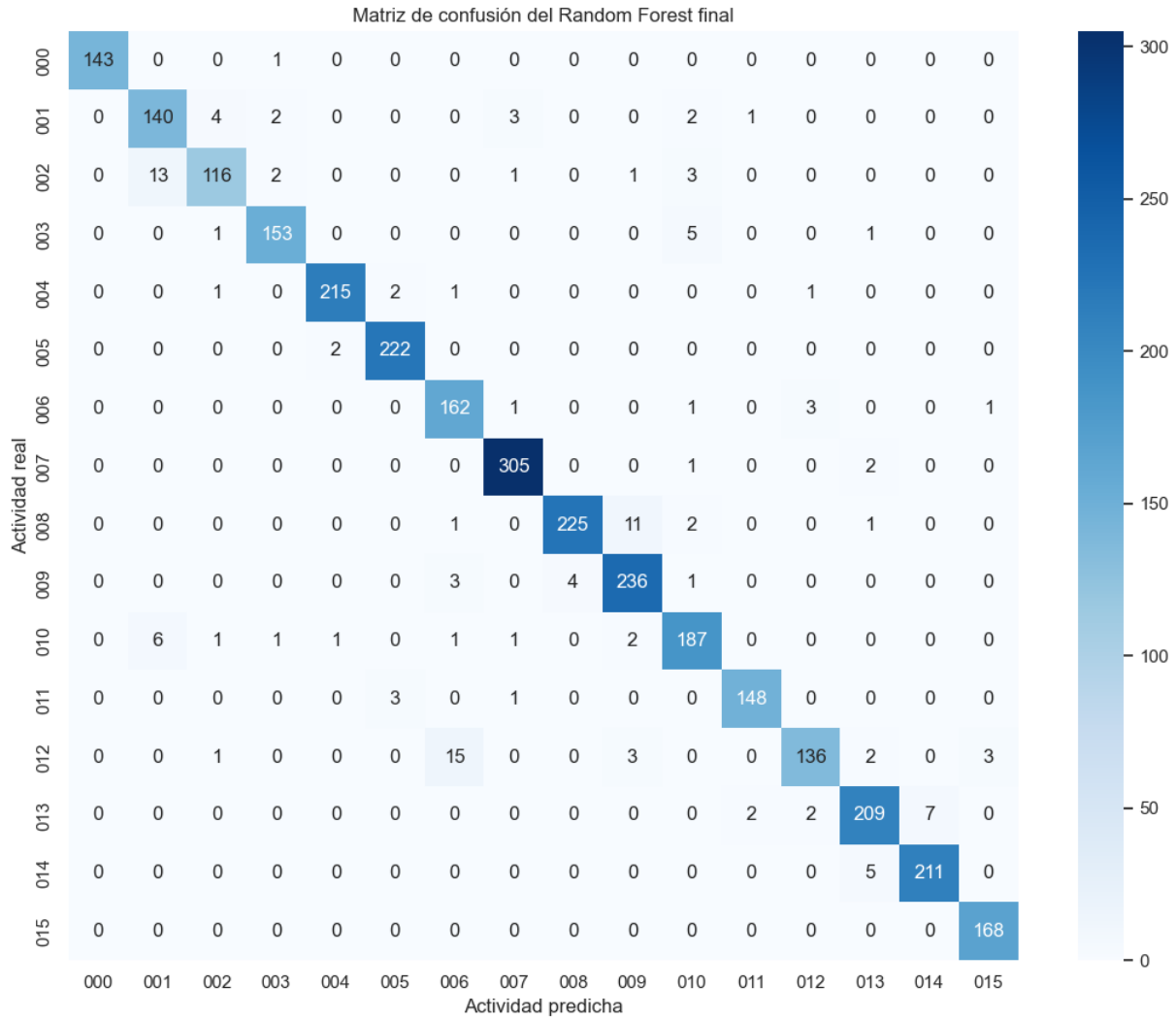


Figura 9: Matriz de confusión del Random Forest definitivo sobre test.

La concentración de valores en la diagonal muestra que la mayoría de las ventanas fue clasificada correctamente. La actividad 000 alcanzó F1 de 0.9965. Las clases que requieren mayor atención son 002, con F1 de 0.8923, y 012, con recall de 0.8500. Estas confusiones pueden relacionarse con patrones cinemáticos similares entre algunos movimientos.

Resultado principal. El modelo definitivo, entrenado con train + validation, obtuvo 95.63% de exactitud, 95.49% de precisión macro y 95.31% de F1 macro en test.

9 Interfaz web

Se desarrolló una interfaz en Streamlit para demostrar el funcionamiento del modelo. Permite seleccionar ejemplos del dataset o cargar dos archivos `.npz` compatibles con los grupos de sensores. La aplicación une los grupos, valida la forma, crea las cuatro ventanas, extrae las 120 características y presenta la predicción de cada ventana.

Para evitar confundir probabilidades internas con la exactitud del sistema, la versión final muestra:

- estructura de entrada y número de ventanas;

- actividad predicha por ventana;
- actividad dominante como resumen visual;
- gráfica interactiva de los canales;
- hiperparámetros definitivos;
- métricas finales obtenidas en test.

La carga manual está pensada para datos nuevos procesados compatibles con REHAB: cada grupo debe tener forma $(880, 6)$ o $(n, 880, 6)$, conservar el mismo orden de canales y haber recibido el mismo filtrado y normalización que los datos de entrenamiento.

10 Limitaciones

- Los archivos procesados no contienen identificadores de paciente o sesión por repetición. No fue posible realizar una separación independiente por sujeto.
- Los resultados representan la clasificación de nuevas ventanas dentro de la población combinada y pueden sobreestimar la generalización a pacientes completamente nuevos.
- Las cuatro ventanas de una repetición comparten etiqueta, aunque una actividad puede contener fases internas diferentes.
- Se utilizó un conjunto fijo de validación. Los resultados de selección pueden depender de esta partición más que con validación cruzada.
- La interpretación está limitada a las etiquetas codificadas 000–015; una implementación clínica requeriría nombres clínicos, protocolos de captura y validación externa.

11 Conclusiones

El proyecto produjo un flujo completo y reproducible desde archivos cinemáticos hasta una interfaz de clasificación. El EDA permitió identificar señales en cero, duplicados, variabilidad entre actividades y relaciones entre canales. La reconstrucción de `014_1.npy` permitió conservar las 16 actividades sin sustituir información con estimaciones arbitrarias.

La separación estratificada por señal completa antes de crear ventanas fue fundamental para evitar fuga de información. La extracción de diez estadísticas por cada uno de los 12 canales convirtió las ventanas en una representación tabular adecuada para modelos clásicos.

Random Forest superó a regresión logística y SVM. La curva de validación mostró un ligero sobreajuste, pero el Grid Search manual de 16 configuraciones confirmó que una regularización más restrictiva no mejoraba el F1 macro de validación. Por ello se conservó la configuración inicial con respaldo experimental.

En síntesis, el Random Forest final obtuvo 95.63% de exactitud, 95.49% de precisión macro y 95.31% de F1 macro sobre test reservado. El siguiente paso recomendado es obtener identificadores de paciente para evaluar generalización por sujeto, además de explorar características temporales o modelos secuenciales sin comprometer la separación por repetición.

12 Contribuciones del equipo

Cuadro 8: Espacio para documentar las contribuciones individuales.

Integrante	Contribución
Integrante 1	Análisis exploratorio, visualizaciones e interpretación de calidad de datos.
Integrante 2	ETL, limpieza, partición agrupada, ventanas y extracción de características.
Integrante 3	Modelación, evaluación, refinamiento e interfaz web.
Trabajo compartido	Reconstrucción del archivo corrupto, revisión de decisiones y elaboración del reporte.

A Reproducibilidad

El repositorio contiene los notebooks y scripts necesarios. El orden recomendado es:

1. ejecutar `ETL/etl_definitivo.ipynb`;
2. ejecutar `modelado/modelacion_3_modelos_def.ipynb`;
3. instalar `Interfaz/requirements.txt`;
4. iniciar la aplicación con `python -m streamlit run Interfaz/app.py`.

El ETL genera `datos_train.csv`, `datos_validation.csv` y `datos_test.csv`. El modelo serializado se encuentra en `Interfaz/modelo_random_forest.joblib`.

B Referencia de formas

Cuadro 9: Transformación de dimensiones a lo largo del pipeline.

Etapas	Forma
Archivo de un grupo	$(n, 880, 6)$
Repetición de un grupo	$(880, 6)$
Repetición combinada	$(880, 12)$
Cuatro ventanas por repetición	$(4, 220, 12)$
Características por repetición	$(4, 120)$
Tabla de entrenamiento	$(9336, 121)$, incluyendo etiqueta
Tabla de validación	$(3112, 121)$, incluyendo etiqueta
Tabla de prueba	$(3112, 121)$, incluyendo etiqueta

Referencias

- [1] M. Lv, Y. Gao, G. Cheng, L. Wang, M. Cai, W. Zhou, L. Guo y K. He, “A wearable sensor-based kinematic dataset collected under standardized rehabilitation tasks from 120 post-stroke patients,” *Scientific Data*, vol. 13, art. 1136, 2026. <https://doi.org/10.1038/s41597-026-07802-2>.

- [2] P. Chapman et al., *CRISP-DM 1.0: Step-by-step data mining guide*, SPSS, 2000.
- [3] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.